EmojiLang

A Visual, Emoji-Based Programming Language

Student Names: Faiq (23L-0753) and Hasan (23I-0703)

1. Purpose of the Language

EmojiLang is a domain-specific programming language for educational programming and compiler pedagogy. It targets non-native English speakers and children learning to code, where English keywords like while, return, and function create an unnecessary cognitive barrier. By replacing keywords with universally recognizable pictographs, EmojiLang reduces syntax anxiety while remaining fully compilable to LLVM IR. It supports variables, arrays, loops, functions, conditionals, and recursion — enough to implement algorithms like Merge Sort and Insertion Sort. It is a proof of concept that syntax can be both novel and functional. Target: Both JIT and AOT.

2. Key Features

Feature / Construct	Emoji	Description
Variable Declaration	🔢	Declares an integer variable.
Assignment	📥	Assigns a value to an existing variable.
If Condition	🤔	Begins an if block.
Else Condition	😢	Begins an else block.
Loop	🔁	Repeats a block for a range or while condition.
Function Definition	⚙️	Declares a function with parameters.
Return	↩	Returns a value from a function.

Feature / Construct	Emoji	Description
Print	📢	Outputs a value to the console.
Array	📦	Declares an array literal.
Index Access	🔍	Accesses an array element by index.
Length	📏	Returns the length of an array.
Slice	✂️	Returns a sub-array from a start index to an end index.
Arithmetic	➕➖ ✖️➗	Basic math operators.
Comparison	⚖️	Comparison operators (>, <, ==, etc.).
Block Delimiters	{}	Curly braces enclose blocks for clarity.
Comments	💬	Single-line comments.

Why It Is Novel

No mainstream compiled language uses emojis as reserved keywords. EmojiLang proves a language can be visually playful while still being formally specified with Flex, Bison, and LLVM.

3. A Small Example

Here is an implementation of Merge Sort written in EmojiLang:

```
None
💬 Merge Sort in EmojiLang


⚙️ merge(left, right) {
```

```
    🔢 result = 📦[]
    🔢 i = 0
    🔢 j = 0

    🔁 i < 📏(left) && j < 📏(right) {
        🤔 🔍(left, i) <= 🔍(right, j) {
            📥 result = 📦[🔍(left, i)]
            i = i + 1
        } 😢 {
            📥 result = 📦[🔍(right, j)]
            j = j + 1
        }
    }

    🔁 i < 📏(left) {
        📥 result = 📦[🔍(left, i)]
        i = i + 1
    }

    🔁 j < 📏(right) {
        📥 result = 📦[🔍(right, j)]
        j = j + 1
    }

    ↩️ result
}

⚙️ merge_sort(arr) {
```

```
    🤔 📏(arr) <= 1 {
        ↩️ arr
    }

    🔢 mid = 📏(arr) / 2
    🔢 left = merge_sort(✂️(arr, 0, mid))
    🔢 right = merge_sort(✂️(arr, mid, 📏(arr)))

    ↩️ merge(left, right)
}

💬 Main program
🔢 data = 📦[38, 27, 43, 3, 9, 82, 10]
🔢 sorted = merge_sort(data)
📤 sorted

Expected Output:[3, 9, 10, 27, 38, 43, 82]
```

This example proves that EmojiLang supports arrays, loops, functions, recursion, conditionals, variables, and return values — enough to implement Merge Sort and Insertion Sort.

4. Requirement Coverage

Requirement	Implementation Details
Flex for lexical analysis	Lexer recognizes emoji tokens, integers, identifiers, and punctuation.
Bison for parsing	Grammar rules define declarations, conditionals, loops, functions, arrays, and expressions.

Requirement	Implementation Details
LLVM IR generation	AST is walked to emit LLVM IR using the LLVM C++ API.
JIT or AOT	JIT via LLVM ExecutionEngine for demo; AOT via object code emission and linking.
Original concepts	No mainstream compiled language uses emojis as reserved keywords.
Loops	🔁
Functions	⚙️
Arrays	📦 + 🔍 + 📏 + ✂️
Variables	🔢
If/else	🤔 / 😢
Can implement Merge Sort	Yes — shown in Section 3.